

Efficient Claude Code

Efficient Context Engineering with Agent Teams

| When Not To Use Agents

Use Deterministic Code When...

- Inputs are well typed
- Failure is expensive
- You will run it 100 times

Use Agent When...

- Input is fuzzy
- Failure is reversible
- You will run it once or a few times only

The probabilistic-deterministic sandwich

Deterministic spec

tests · types · schemas · contracts

Agent (probabilistic)

explores the solution space

Deterministic validator

tests pass · contract honored

Continuous Autonomy

Deploying agents as long-running, unattended engineering workers.

| Unattended Execution & Loops

Headless Daemon Mode

Run Claude Code as a background process using `claude -p "Audit repo"`. Use `--allowedTools` to lock down capabilities for safer autonomous runs.

Guardrails & Safety

Prevent runaway costs with `--max-budget-usd` and `--max-turns`. Claude will automatically pause and checkpoint if limits are approached.

Project Isolation & Context

Maximizing signal, minimizing noise, and ensuring safety.

The "Context Poisoning" Problem

Signal Decay

Unmanaged context leads to token pollution, where extraneous file content dilutes the agent's reasoning. This results in hallucinations and "context bleed" between unrelated project modules.

Logical Contamination

Without strict isolation, an agent may incorrectly apply patterns from Project A to the codebase of Project B, leading to architectural drift and subtle, hard-to-debug logic errors.

CLAUDE.md: The North Star

Every isolated project should feature a CLAUDE.md file, a persistent, human-readable state registry.

It serves as the project's "memory bank," storing architectural decisions, naming conventions, and build commands that would otherwise be lost when the agent's session context is cleared.



The 60% Compaction Rule

60%

Token Utilization Trigger

Proactive Optimization

When an agent session reaches 60% of its available context window, reasoning quality begins to degrade. Teams should implement manual or automatic **context compaction**, resetting the window while carrying over critical state summaries to maintain peak cognitive performance.

Slash commands turn habits into named tools

.claude/commands/

- |— pr-review.md # Reviews current branch as if I were on-call
- |— pre-merge-checks.md # Runs my mental "is this ready" checklist
- |— triage-flake.md # Repro-and-diagnose for flaky tests
- |— add-tracing.md # Adds structured logs/spans to repo conventions
- |— runbook-update.md # When fixing a recurrent issue, also update runbook
- |— repro-from-trace.md # Generate a minimal repro from Sentry/Datadog

Agent Teams

Orchestrating Parallel Engineering Workflows with Agent Teams

The Evolution of AI Coding



Solo Agent

Single model session handling one task at a time. Context becomes diluted as complexity grows.



Sub-Agents

Hierarchical reporting where workers return results to the lead. Minimal peer interaction.



Agent Teams

Autonomous specialists working in parallel with peer-to-peer messaging and shared task lists.

Multi-Agent Swarm Architecture



Team Lead

The central orchestrator responsible for high-level planning, task decomposition, and final conflict resolution between sub-agents.



Domain Specialists

Focused instances (e.g., Security Auditor, Frontend Engineer) running in isolated 1M context windows for deep technical focus.



Peer Communication

Asynchronous "mailbox" messaging allows specialists to resolve dependencies directly without bottlenecking the Team Lead.

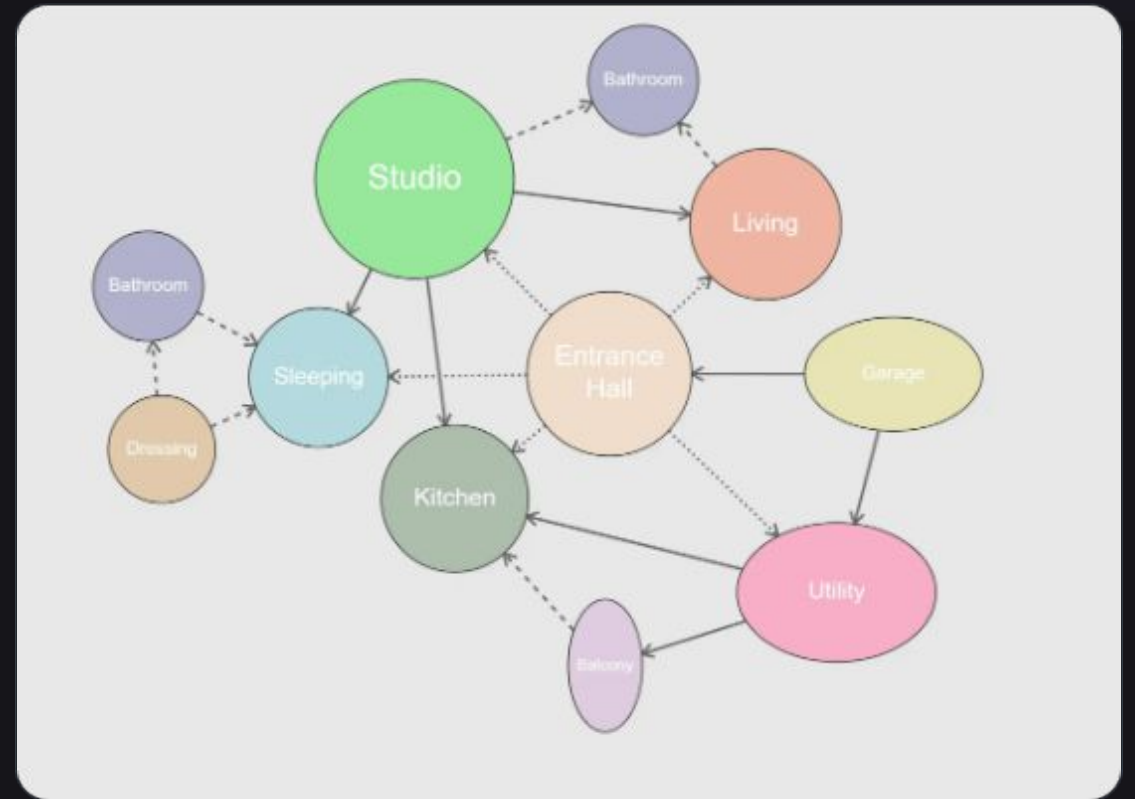
The Supervisor-Specialist Model

The architecture shifts from linear execution to a robust multi-agent orchestration:

Team Lead: Orchestrates context, breaks down tasks, and performs final synthesis of results.

Specialist Agents: Full Claude Code instances with isolated context windows (1M tokens) for deep focus.

Peer Communication: Direct "mailbox" messaging allows agents to resolve dependencies without the Lead.



Seven Core Orchestration Primitives

TeamCreate

Initializes team resources and environment.

TaskCreate

Defines units of work and dependencies.

TaskUpdate

Claims, starts, or completes assignments.

TaskList

Monitors global status across the swarm.

TaskGet

Retrieves deep details for specific workers.

SendMessage

DMs or broadcasts to teammates.

Task (Spawn)

Launches a new specialist instance.

TeamDelete

Cleans up resources and saves logs.

The tmux Advantage

Traditional terminals flatten agent outputs into a single stream. **tmux** unlocks true multi-pane observability.

- Watch independent workstreams in real-time.
- Inspect per-agent command history and logs.
- **Intervention:** Switch panes to stop or redirect an agent mid-task.
- Persistence: Tmux sessions survive terminal crashes.

```
.
ss+:`
yssss+-
MNysssso.
MMhssssss/
dyssssssss+
MNhssssssss/
MMMdssssssss.
VMMMyssssssss+
hmmmhssssssso
hmmmhssssssso
VMMMyssssssss+
MMMdssssssss.
MNhssssssss/
dyssssssss+
MMhssssss/
MNysssso.
yssss+-
ss+:`
.
```

ham@hambook-2

OS: Ubuntu 23.10 x86_64

Host: XPS 13 9300

Kernel: 6.5.0-14-generic

Uptime: 48 mins

Packages: 2744 (dpkg), 47 (nix-default)

Shell: zsh 5.9

Resolution: 3440x1440

DE: GNOME 45.1

WM: Mutter

WM Theme: Adwaita

Theme: Yaru-red-dark [GTK2/3]

Icons: Yaru-red-dark [GTK2/3]

Terminal: tmux

CPU: Intel i7-1065G7 (8) @ 3.900GHz

GPU: Intel Iris Plus Graphics G7

Memory: 6853MiB / 15564MiB

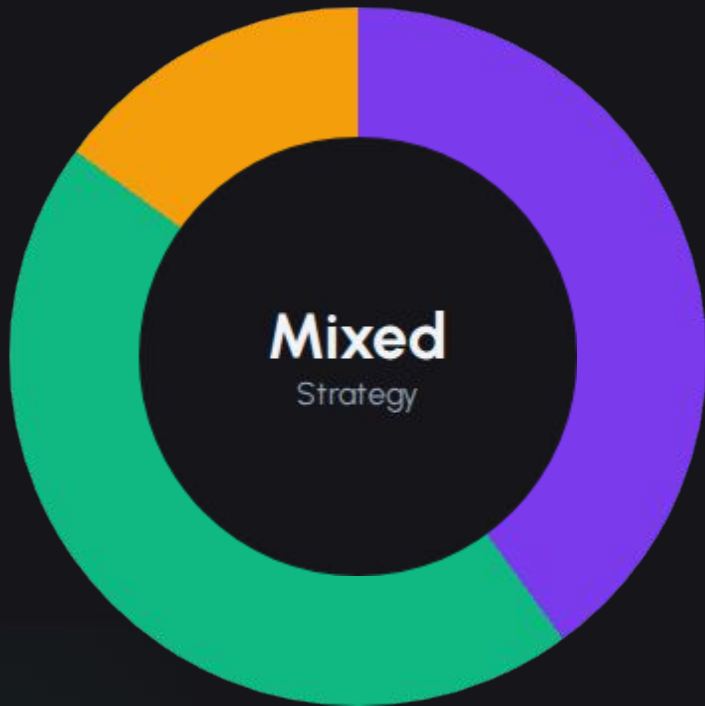


```
→ .dotfiles
alacritty nix
bat nvim
foot oh-
git REA
gnupg scr
i3 ssh
i3status swa
→ .dotfiles
```

	File:
1	dotfi
2	=====
3	
4	A col
	les.
5	
6	Here'
	t loo
	ppucc
	/catp

:[]

Strategic Model Allocation

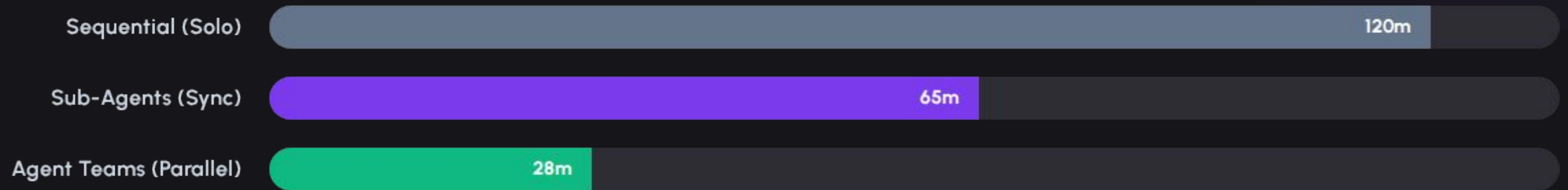


Opus 4.6 (Lead): Complex reasoning, architectural decisions, and conflict resolution.

Sonnet 3.5 (Workers): Heavy-duty implementation, file editing, and command execution.

Haiku 3.5 (Reviewers): Fast unit test generation, formatting, and routine security scans.

| Impact on Engineering Throughput



Wall-clock time reduction based on a 15-file refactor task involving 4 specialized domains.

Safe Multi-Agent Collaboration



Git-Based Locking

Agents claim tasks by writing lock files to `.claude/tasks/`.
This prevents overlapping edits on the same files.



The Mailbox System

Messaging happens asynchronously. An agent can "Ping" a security specialist to verify a fix before committing.

Claude resolves merge conflicts automatically by rolling back and retrying with updated context.

High-Value Swarm Use Cases



Parallel Security Audit

Deploy 3 agents: one for SQLi, one for Auth logic, one for PII exposure. Run them simultaneously across the repo.



Large Scale Migration

Migrating from JS to TS or Jest to Vitest. Assign sub-folders to different teammates for parallel conversion.



Heuristic Debugging

Agents explore different hypothesis paths (Database, API, Frontend) to find the root cause of a race condition.

| Enabling Team Mode

1. Enable Flag

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

2. Configure Tmux Mode

Update ~/.claude/settings.json:

```
{  
  "teammateMode": "tmux",  
  "defaultModel": "claude-3-5-sonnet-20241022"  
}
```

>_ CLI Entry

Start with explicit teammate mode:

```
claude --teammate-mode tmux
```

Note: Ensure you are already inside a tmux session for automatic pane splitting.

Build Tic-Tac-Toe with Agent Teams